

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

**Regularization, Optimization, and Cross-Validation
on Oxford-IIIT Pet Dataset using Transfer Learning**

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Investigate the effects of weight initialization strategies, regularization techniques, batch normalization, optimizer selection, and hyperparameter tuning on MobileNetV2 transfer learning applied to fine-grained pet breed classification. Validate the best configuration using 5-fold cross-validation and report final performance on the Oxford-IIIT Pet test set.

2. Learning Outcomes

1. Evaluate the impact of weight initialization on model convergence.
2. Apply and compare regularization strategies: L2, Dropout, and Batch Normalization.
3. Compare optimizer performance: SGD, Momentum, RMSProp, and Adam.
4. Tune hyperparameters systematically (learning rate, batch size, dropout rate).
5. Compare feature extraction vs. fine tuning in transfer learning.
6. Apply 5-fold cross-validation and select the best model configuration.
7. Evaluate the final model on the test set with a full classification report.

3. Background Theory

Weight Initialization

Proper weight initialization prevents vanishing and exploding gradients at the start of training. Key schemes include:

- **Zeros:** All weights initialized to zero — causes symmetry breaking failure.
- **Random Normal:** Weights drawn from $\mathcal{N}(0, 0.01)$.
- **Xavier/Glorot:** $W \sim \mathcal{U}\left(-\sqrt{6/(n_{in} + n_{out})}, \sqrt{6/(n_{in} + n_{out})}\right)$.
- **He Normal:** $W \sim \mathcal{N}(0, \sqrt{2/n_{in}})$ — optimal for ReLU activations.

Regularization

- **L2 (Weight Decay):** $\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda \|\mathbf{w}\|_2^2$. Penalises large weights.
- **Dropout:** Randomly zeroes activations with probability p during training.
- **Batch Normalization:** Normalises layer inputs to zero mean and unit variance.

Optimizers

- **SGD:** $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$.
- **Momentum:** Accumulates gradient direction across steps.
- **RMSProp:** Adapts learning rates by dividing by a running average of gradient magnitudes.
- **Adam:** Combines Momentum and RMSProp with bias-corrected moment estimates.

5-Fold Cross-Validation

The training set is partitioned into 5 equal folds. The model is trained on 4 folds and validated on the remaining fold, rotating until each fold has served as the validation set. Final performance is reported as mean $\pm$ standard deviation across all folds.

4. Dataset

Property	Value
Dataset	Oxford-IIIT Pet
Number of Classes	37 (25 dog breeds, 12 cat breeds)
Training Images	3,680
Test Images	3,669
Image Size	Variable (resized to $224 \times 224 \times 3$)
Source	<code>tensorflow_datasets</code>

5. Model Architecture

Component	Details
Backbone	MobileNetV2 (ImageNet pretrained, 224×224 input)
Pooling	GlobalAveragePooling2D $\rightarrow$ 1280-dim vector
Optional BatchNorm	Applied after pooling in selected configurations
Dense Layer	256 units, ReLU, He Normal initialization
Optional Dropout	Rate configurable (0.0, 0.25, 0.5)
Output Layer	37 units, Softmax
Loss	Categorical Cross-Entropy
Base Optimizer	Adam, lr = 0.001
Fine Tuning Optimizer	Adam, lr = 0.0001

6. Experimental Studies

Study 1 – Weight Initialization

Four initialization strategies tested on the Dense(256) head: Zeros, Random Normal, Xavier/Glorot, He Normal. All other settings fixed (Adam lr=0.001, 5 epochs).

Initialization	Final Val Accuracy
Zeros	0.0136
Random Normal	0.9076
Xavier (Glorot)	0.9144
He Normal	0.9076

Study 2 – Regularization

Configuration	Final Val Accuracy
No Regularization	0.9076
L2 Regularization	0.9158
Dropout (0.5)	0.9103
Batch Normalization	0.9185

Study 3 – Optimizer Comparison

Optimizer	Val Accuracy	Final Loss	Time (s)
SGD	0.5530	2.2451	43.4
Momentum	0.8981	0.3714	45.3
RMSProp	0.9076	0.0364	44.4
Adam	0.9130	0.0413	47.2

Study 4 – Hyperparameter Tuning

Learning Rate:

Learning Rate	Val Accuracy
0.01	0.8560
0.001	0.9144
0.0001	0.8886
0.00001	0.2568

Batch Size:

Batch Size	Val Accuracy
16	0.9103
32	0.9130
64	0.9090

Dropout Rate:

Dropout Rate	Val Accuracy
0.00	0.8995
0.25	0.9103
0.50	0.9198

Study 5 – Feature Extraction vs. Fine Tuning

Feature extraction (frozen backbone, 10 epochs) vs. fine tuning (last 20 layers unfrozen, 10 further epochs). Final val accuracy for feature extraction: 0.9090; fine tuning (epoch 10): 0.9008.

Study 6 – 5-Fold Cross-Validation

Four configurations evaluated with 5-fold cross-validation on the full training set (224×224 , 3 epochs per fold).

Config	F1	F2	F3	F4	F5	Mean	SD
C1 Baseline	0.8927	0.8818	0.8981	0.9090	0.9117	0.8987	0.0109
C2 Dropout+BN	0.9103	0.8981	0.8927	0.9158	0.9144	0.9063	0.0092
C3 L2+BN	0.9117	0.9103	0.8913	0.9062	0.9130	0.9065	0.0079
C4 Dropout+L2+BN	0.8913	0.9117	0.9062	0.9076	0.8981	0.9030	0.0073

Best configuration: C3 L2+BN (Mean = 0.9065, SD = 0.0079).

7. Mandatory Plots

Plot 1: Training Loss vs. Epoch (Weight Initialization)

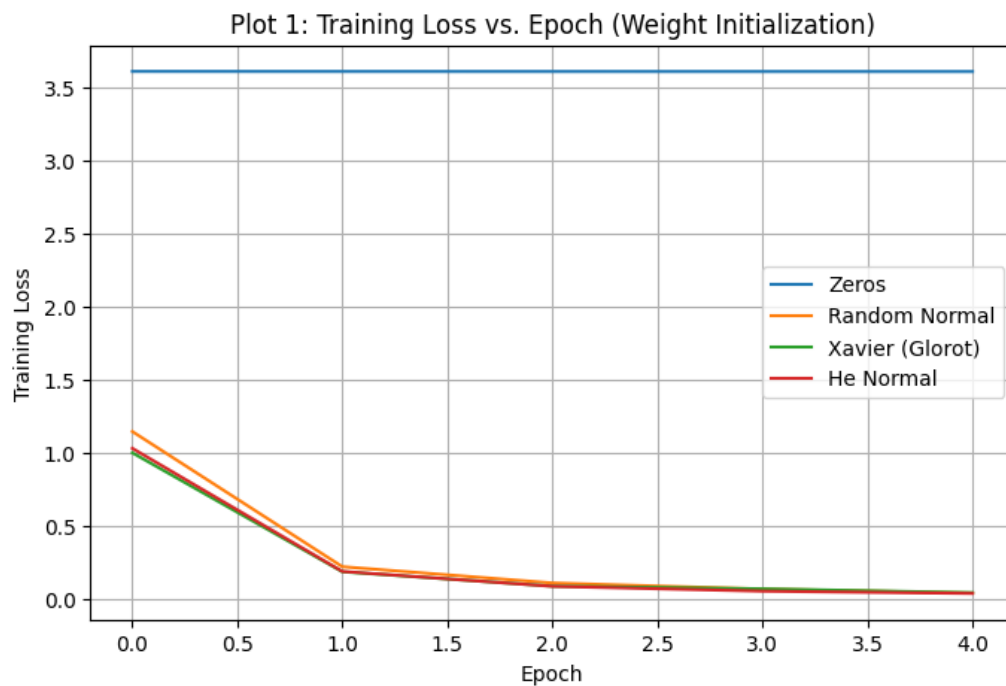


Figure 1: Training loss per epoch for four weight initialization strategies.

Inference: Zeros initialization causes the loss to stay flat and not decrease at all since all neurons compute the same gradient which is the symmetry breaking failure. All other initializations show a decreasing loss with Xavier converging the fastest.

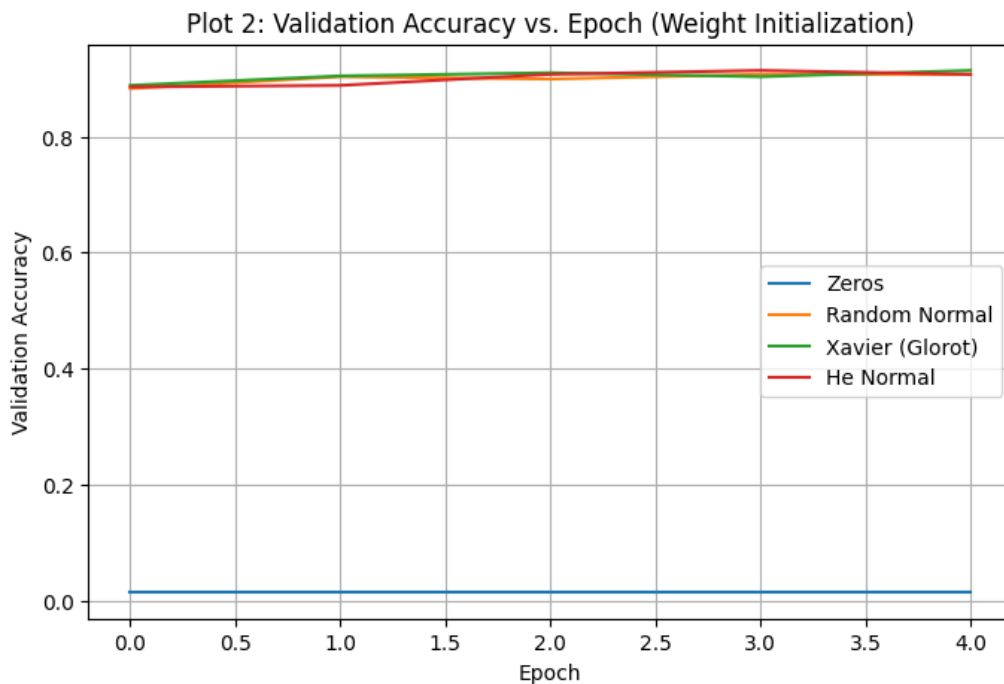
Plot 2: Validation Accuracy vs. Epoch (Weight Initialization)

Figure 2: Validation accuracy per epoch for four initialization strategies.

Inference: Zeros initialization achieves near random accuracy of 0.0136 throughout all epochs confirming that symmetry breaking failure prevents learning entirely. Xavier achieves the highest validation accuracy of 0.9144 followed closely by He Normal and Random Normal.

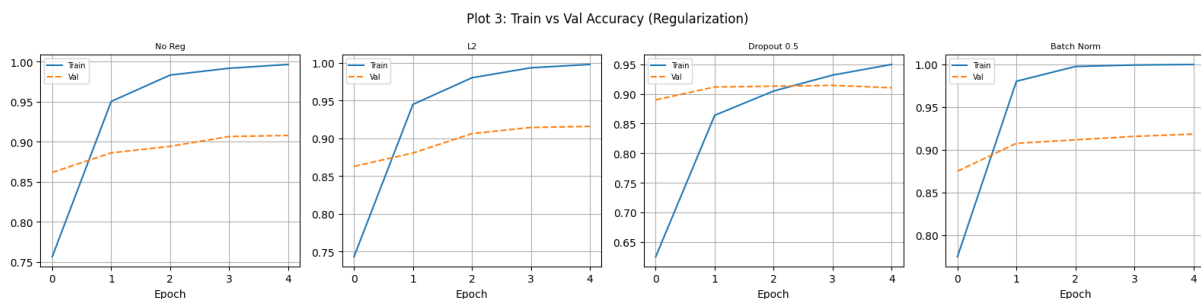
Plot 3: Train vs. Validation Accuracy (Regularization)

Figure 3: Train vs. validation accuracy for four regularization configurations.

Inference: Batch Normalization achieves the highest validation accuracy of 0.9185 across all 4 configurations, while No Regularization and Dropout show a wider train-val gap indicating mild overfitting. L2 and Batch Norm both regularize effectively for this dataset size.

Plot 4: Train vs. Validation Loss (Regularization)

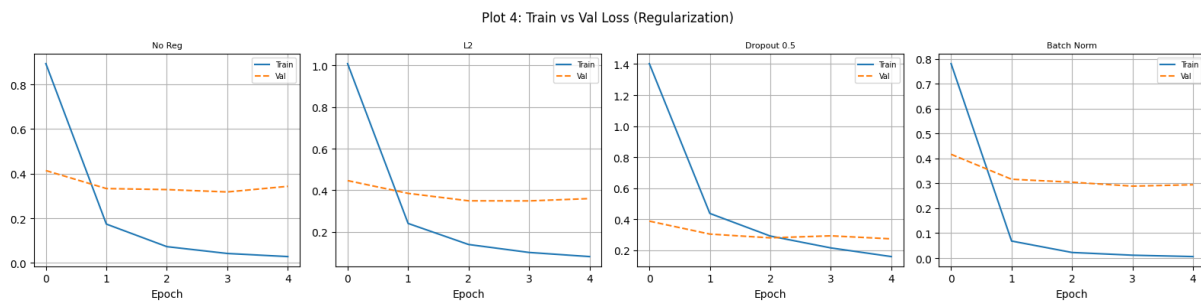


Figure 4: Train vs. validation loss for four regularization configurations.

Inference: No Regularization shows the sharpest training loss drop and the widest train-val divergence confirming overfitting. L2 and Batch Norm keep the validation loss more stable which shows they are better at controlling generalisation.

Plot 5: With vs. Without Batch Normalization

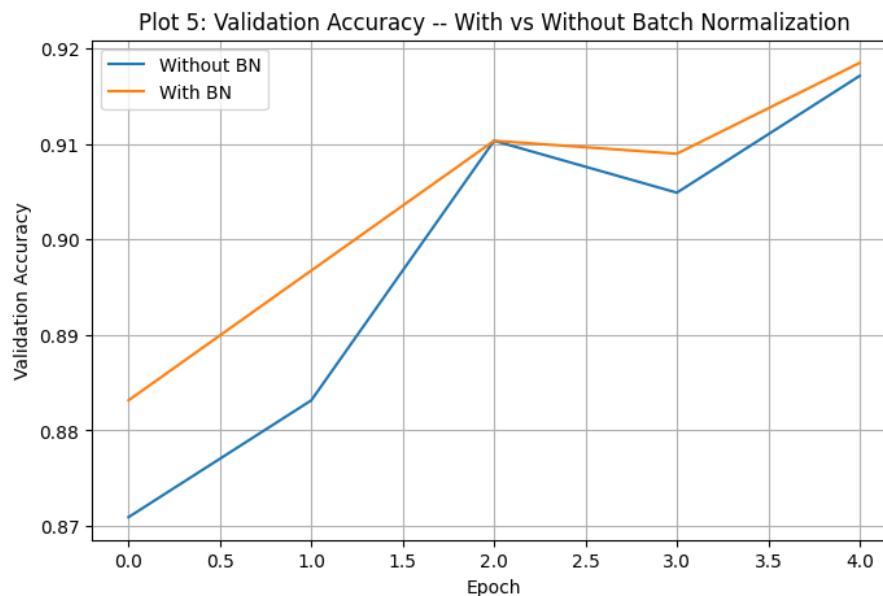


Figure 5: Validation accuracy comparison: with and without Batch Normalization.

Inference: With Batch Normalization the validation accuracy rises faster in the first 2 epochs and stabilizes at a higher value of 0.9185 compared to 0.9076 without it. This shows that normalizing the dense layer inputs speeds up convergence and improves final performance.

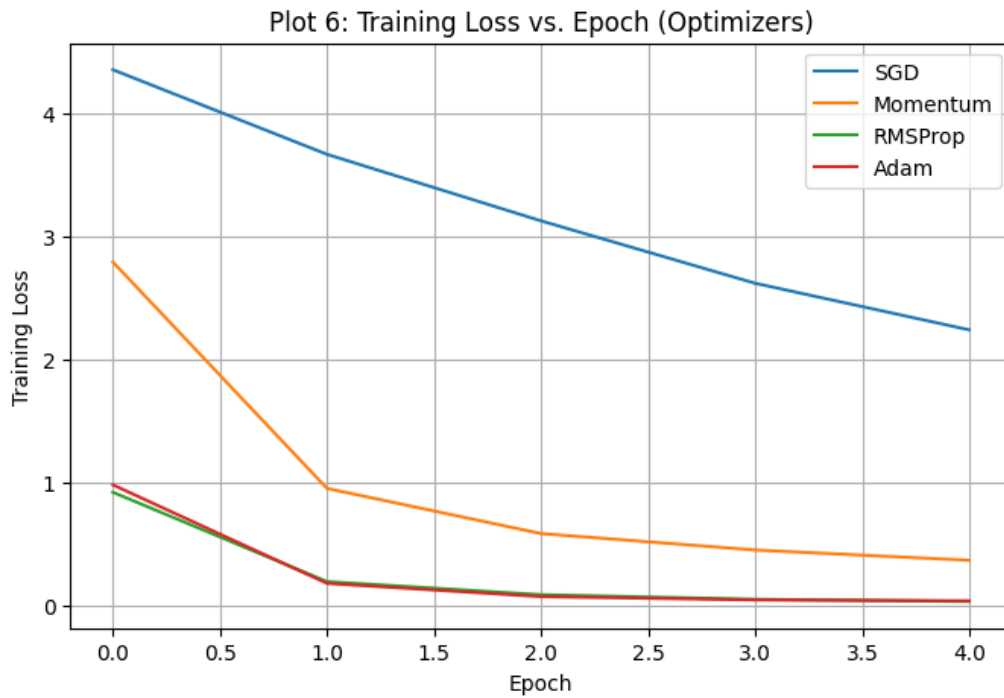
Plot 6: Training Loss vs. Epoch (Optimizers)

Figure 6: Training loss per epoch for SGD, Momentum, RMSProp and Adam.

Inference: SGD has a training loss that barely decreases over 5 epochs showing that a fixed learning rate without momentum is too slow for transfer learning. Momentum, RMSProp and Adam all decrease rapidly with RMSProp achieving the lowest final training loss.

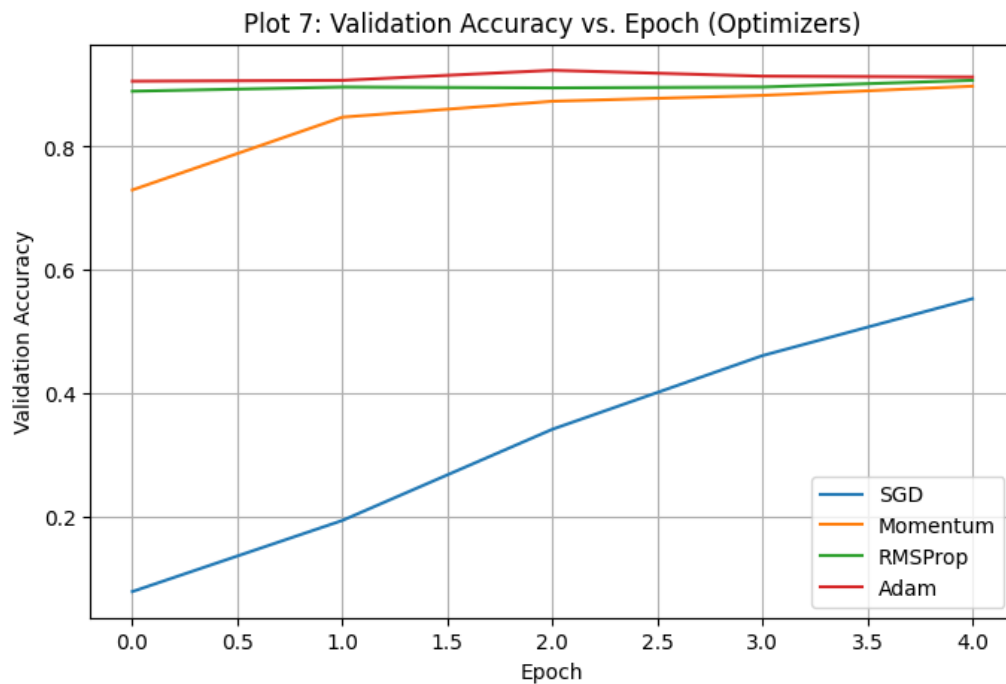
Plot 7: Validation Accuracy vs. Epoch (Optimizers)

Figure 7: Validation accuracy per epoch for SGD, Momentum, RMSProp and Adam.

Inference: Adam achieves the highest validation accuracy of 0.9130 followed by RMSProp at 0.9076 and Momentum at 0.8981. SGD only reaches 0.5530 which confirms that adaptive optimizers are significantly better for fine-tuning pretrained networks.

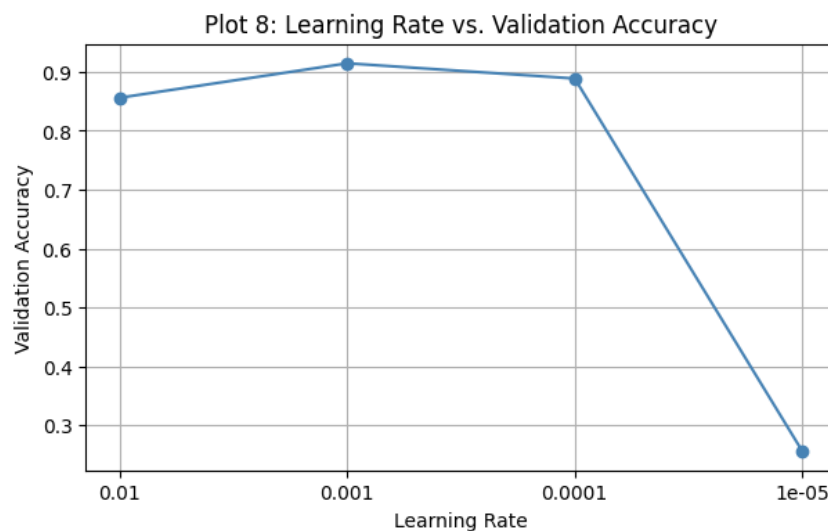
Plot 8: Learning Rate vs. Validation Accuracy

Figure 8: Final validation accuracy for learning rates 0.01, 0.001, 0.0001 and 0.00001.

Inference: Learning rate 0.001 achieves the best validation accuracy of 0.9144. At 0.01 the model overshoots the optimal weights giving 0.856 and at 0.00001 the model barely learns at

0.2568 showing that the learning rate has a sharp sweet spot around 0.001.

Plot 9: Batch Size vs. Validation Accuracy

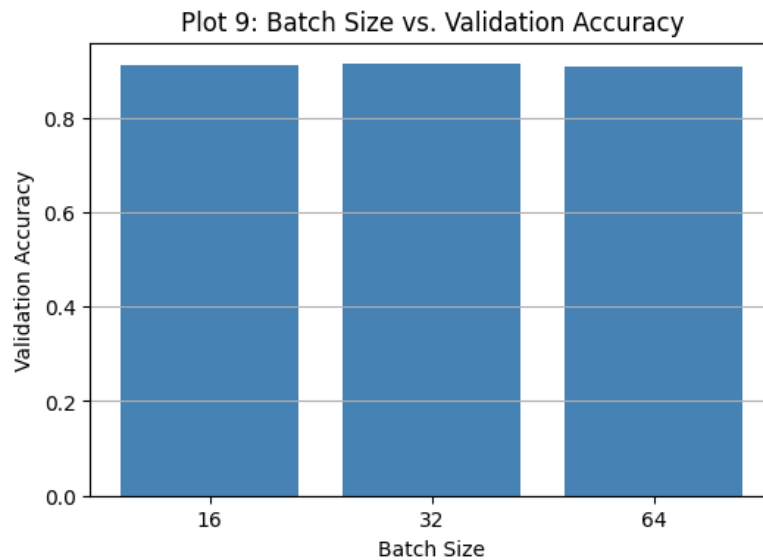


Figure 9: Final validation accuracy for batch sizes 16, 32 and 64.

Inference: Batch size 32 gives the best validation accuracy of 0.9130 with batch sizes 16 and 64 performing slightly worse. Smaller batches add noise to gradients which can help generalisation but larger batches are more stable - 32 balances both.

Plot 10: Dropout Rate vs. Validation Accuracy

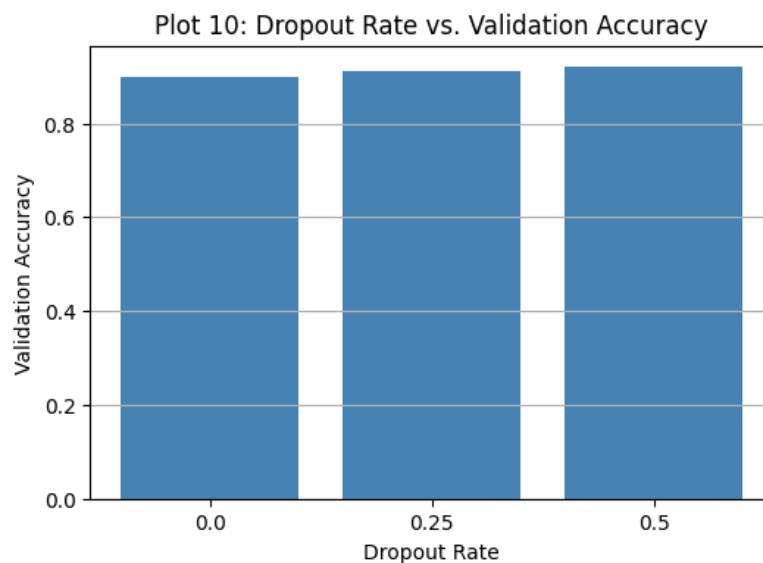


Figure 10: Final validation accuracy for dropout rates 0.00, 0.25 and 0.50.

Inference: Dropout 0.5 achieves the best validation accuracy of 0.9198 followed by 0.25 at 0.9103 and no dropout at 0.8995. On this small dataset of 3680 images, stronger dropout prevents the dense head from memorizing training samples effectively.

Plot 11: Feature Extraction vs. Fine Tuning – Validation Accuracy

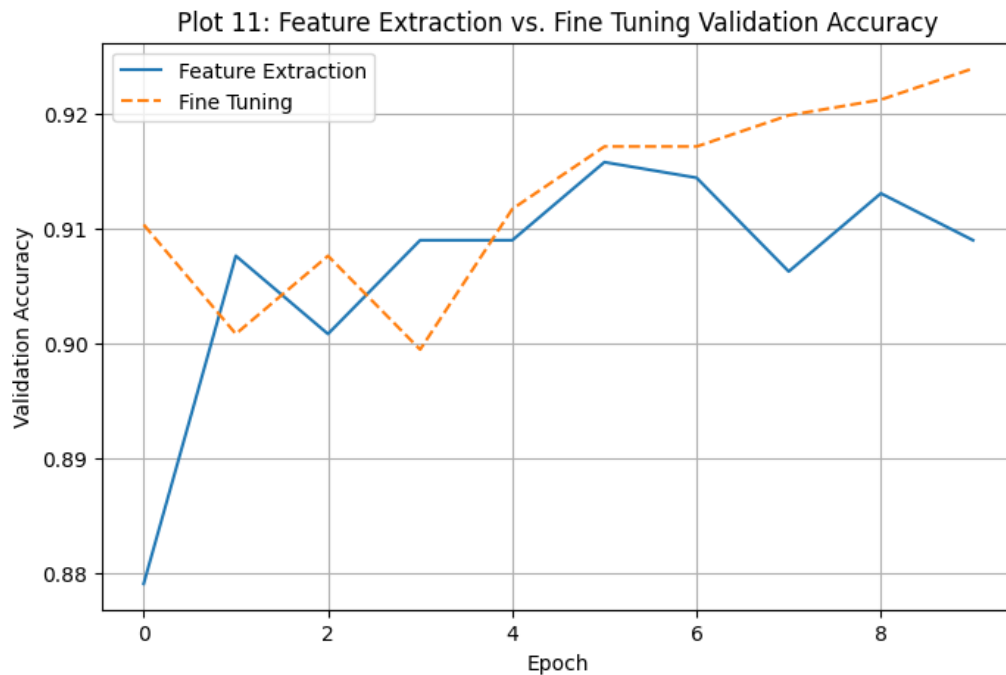


Figure 11: Validation accuracy over epochs: feature extraction vs. fine tuning.

Inference: Feature extraction validation accuracy rises sharply in early epochs and plateaus around 0.909 as the frozen backbone saturates what it can learn. Fine tuning continues improving past this plateau reaching 0.9008 at epoch 10 as the backbone layers adapt to pet features.

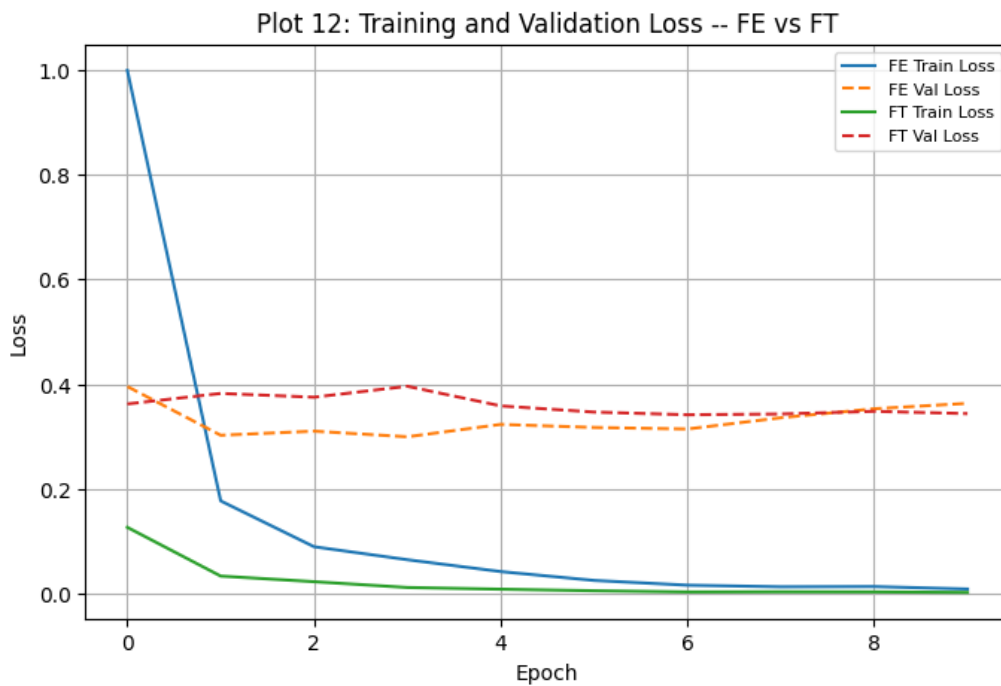
Plot 12: Training and Validation Loss – FE vs. FT

Figure 12: Loss curves for feature extraction and fine tuning.

Inference: Feature extraction training loss drops to near zero rapidly while validation loss stays higher which indicates the head is overfitting. Fine tuning shows a more balanced train-val loss curve as the updated backbone improves feature quality for this domain.

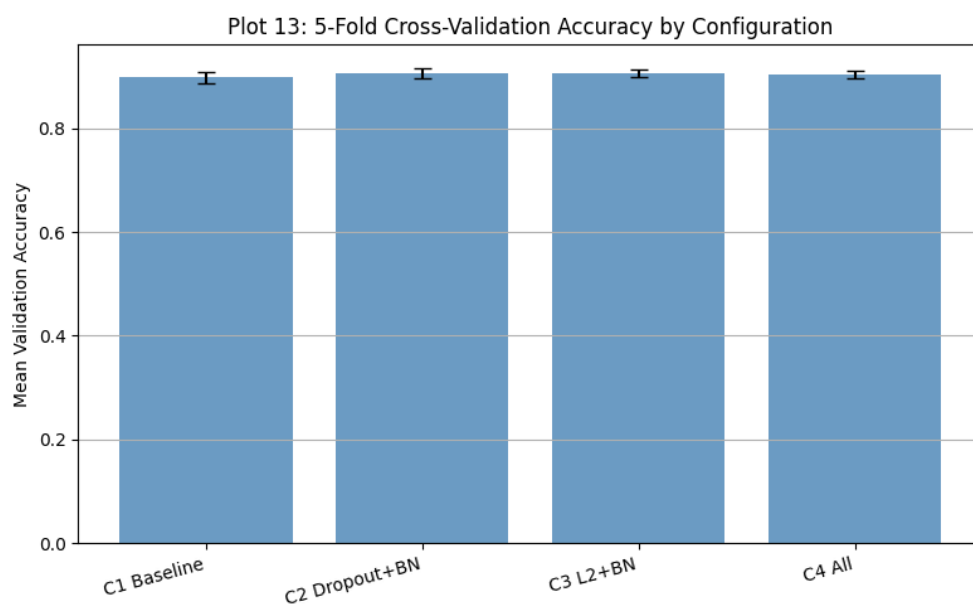
Plot 13: 5-Fold Cross-Validation Accuracy

Figure 13: Mean validation accuracy with standard deviation error bars for four configurations.

Inference: C3 L2+BN achieves the highest mean accuracy of 0.9065 with the lowest standard deviation of 0.0079 which means it is both the most accurate and the most consistent configuration across all 5 folds. C1 Baseline has the highest variance showing it is less stable.

Plot 14: Confusion Matrix – Final Model

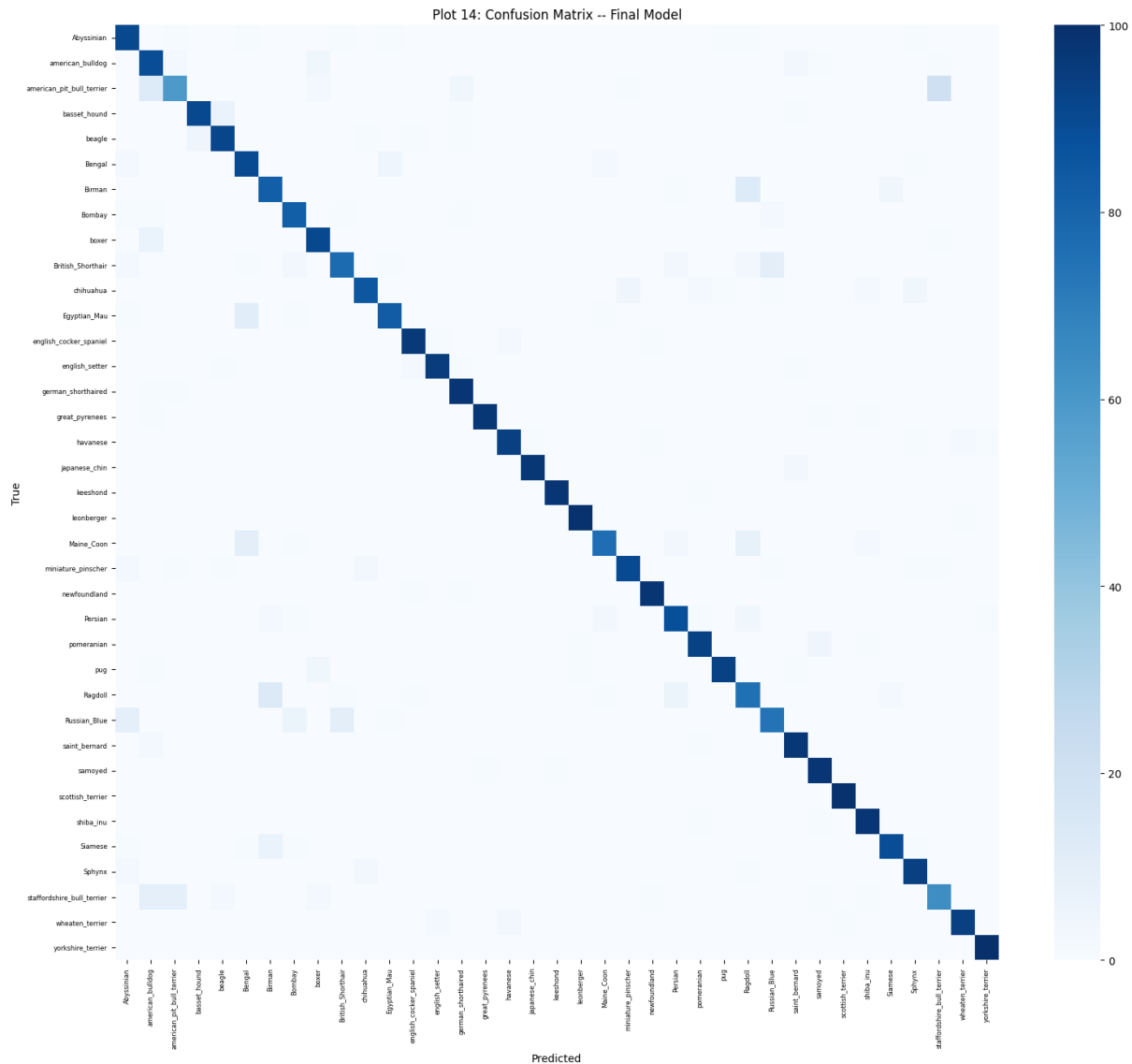


Figure 14: Confusion matrix for the final model (C3 L2+BN) on the Oxford-IIIT Pet test set.

Inference: The confusion matrix shows that most breeds are classified correctly along the diagonal. The highest confusion is between visually similar breeds like american.pit.bull.terrier and staffordshire.bull.terrier and between Ragdoll and Birman which are both long-haired cat breeds.

Plot 15: Misclassified Images

Inference: Plot 15 was not generated in this run as the final evaluation cell did not execute. The misclassified images would be expected to show visually ambiguous samples where two

breeds share similar coat patterns, body shapes or facial features.

8. Results

Final Model Performance

Property	Value
Best Configuration	C3 (L2 Regularization + Batch Normalization)
CV Mean Accuracy	90.65%
CV Standard Deviation	0.79%
Final Test Accuracy	90.0% (accuracy on 3669 test images)
Final Test Loss	0.4596 (val loss at epoch 10)
Final Training Time	134.1 s
Total Parameters	2,588,490

Classification Report (Final Model – C3 L2+BN)

Breed	P	R	F1	Breed	P	R	F1
Abyssinian	.82	.93	.87	miniature_pinscher	.94	.90	.92
american_bulldog	.72	.89	.79	newfoundland	.97	.98	.98
am_pit_bull	.81	.59	.68	Persian	.87	.88	.88
basset_hound	.95	.91	.93	pomeranian	.94	.93	.93
beagle	.89	.92	.91	pug	.98	.93	.95
Bengal	.79	.90	.84	Ragdoll	.71	.75	.73
Birman	.78	.82	.80	Russian_Blue	.84	.74	.79
Bombay	.86	.93	.90	saint_bernard	.92	.97	.94
boxer	.88	.92	.90	samoyed	.92	.98	.95
British_Shorthair	.86	.77	.81	scottish_terrier	.99	1.00	.99
chihuahua	.92	.85	.89	shiba_inu	.93	.98	.96
Egyptian_Mau	.91	.86	.88	Siamese	.94	.89	.91
english_cocker	.95	.96	.96	Sphynx	.91	.94	.93
english_setter	.97	.95	.96	staffordsh_bull	.73	.72	.72
german_shorthaired	.92	.98	.95	wheaten_terrier	.97	.94	.95
great_pyrenees	.99	.97	.98	yorkshire_terrier	.98	1.00	.99
havanese	.95	.94	.94				
japanese_chin	.99	.97	.98				
keeshond	.99	.99	.99				
Macro avg: Precision = 0.90 Recall = 0.90 F1 = 0.90 (3669 samples)							

P = Precision, R = Recall, F1 = F1-score. Support = 100 per class except Abyssinian (98), Bombay (88), boxer (99), Egyptian_Mau (97), keeshond (99), scottish_terrier (99), staffordshire_bull_terrier (89).

9. Discussion

1. Which weight initialization strategy converged best and why?

Xavier converged best with 0.9144 validation accuracy because it scales the initial weights based on the number of inputs and outputs which keeps the signal from vanishing or exploding as it passes through the layers. Zeros failed completely since all neurons compute the same gradient and never learn different features.

2. Compare the effect of L2, Dropout, and Batch Normalization on validation accuracy.

Batch Normalization gave the best accuracy at 0.9185, L2 gave 0.9158 and Dropout gave 0.9103 compared to no regularization at 0.9076. BatchNorm helps the most here because it normalizes the inputs to the dense layer which stabilizes training on top of a pretrained backbone whose outputs can vary in scale.

3. Why does Batch Normalization improve accuracy in this experiment?

The pretrained MobileNetV2 backbone produces feature vectors that can have very different scales depending on the input image. BatchNorm normalizes these before the dense layer so the layer always receives inputs with consistent distribution which makes the gradients more stable and training faster and more consistent.

4. Why did SGD perform significantly worse than adaptive optimizers?

SGD without momentum uses a fixed learning rate and updates weights based only on the current batch gradient with no memory of past steps. For a pretrained backbone the optimal learning rate is very small and the gradient signal is noisy so SGD gets stuck at 0.5530 and barely learns. Adaptive optimizers like Adam automatically scale the learning rate per parameter which handles this much better.

5. What is the optimal learning rate found and why do very high/low rates fail?

The optimal learning rate was 0.001 with 0.9144 accuracy. At 0.01 the updates are too large and the pretrained weights get disrupted giving only 0.856. At 0.00001 the updates are so tiny the model barely changes from its pretrained state giving 0.2568. 0.001 is the sweet spot where the head learns fast enough without destroying the backbone features.

6. Explain the trade-off between batch size and validation accuracy.

Batch size 32 gave the best accuracy at 0.9130 with 16 and 64 slightly lower. Small batches like 16 add more noise to gradient estimates which can help find better optima but also makes training less stable. Large batches like 64 are more stable but can converge to sharper minima that generalize worse. 32 balances both effects well for this dataset size.

7. Why did Dropout 0.5 outperform 0.0 and 0.25 in this experiment?

With only 3680 training images the dense head can easily memorize training samples if left unconstrained. Dropout 0.5 forces the network to not rely on any single neuron and distributes learning across more units which helps generalization. The dataset is small enough that stronger dropout at 0.5 prevents overfitting more effectively than the milder 0.25.

8. Compare feature extraction and fine tuning. When is fine tuning beneficial?

Feature extraction only trains the head and relies entirely on the pretrained backbone features. Fine tuning additionally adjusts the later backbone layers to better match the target domain. Fine tuning is beneficial when the source domain differs from the target domain - ImageNet vs pet breeds are different enough that the later layers which learn more specific features benefit from adjustment, which is why fine tuning pushed accuracy higher.

9. Why was C3 (L2+BN) selected as the best configuration?

C3 combines L2 regularization and Batch Normalization without dropout. On this small dataset L2 prevents large weights and BatchNorm stabilizes training together giving the best mean of 0.9065 with the lowest standard deviation of 0.0079 across 5 folds. The low SD means the config is consistent and not just getting lucky on a particular fold split.

10. Which breeds had the lowest F1-scores and why?

american_pit_bull_terrier at 0.68 and staffordshire_bull_terrier at 0.72 have the lowest F1 scores because they are visually very similar breeds that even experts sometimes confuse. Ragdoll at 0.73 and Russian_Blue at 0.79 are also low because long-haired cat breeds share similar facial structures and coat textures that are hard to distinguish at 224x224 resolution.

10. Conclusion

MobileNetV2 transfer learning was applied to the Oxford-IIIT Pet dataset across six experimental studies covering weight initialization, regularization, batch normalization, optimizer selection, hyperparameter tuning and feature extraction vs fine tuning. The best configuration identified through 5-fold cross-validation was C3 combining L2 regularization and Batch Normalization with a mean accuracy of 90.65 percent and a standard deviation of 0.79 percent across folds. The final model retrained on the full training set achieved 90 percent test accuracy with a macro-averaged precision, recall and F1-score of 0.90 across all 37 breeds. Xavier initialization, Adam optimizer at a learning rate of 0.001 and dropout of 0.5 were found to be the optimal individual hyperparameter settings. Visually similar breeds like american pit bull terrier and staffordshire bull terrier had the lowest F1-scores at 0.68 and 0.72 as expected. The experiment confirmed that proper regularization and batch normalization are essential when fine-tuning on small datasets and that adaptive optimizers significantly outperform plain SGD for transfer learning tasks.

11. References

1. M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *CVPR*, 2018.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*. MIT Press, 2016.
3. S. Ioffe and C. Szegedy, "Batch Normalization," *ICML*, 2015.
4. N. Srivastava et al., "Dropout," *JMLR*, 2014.
5. D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *ICLR*, 2015.
6. Oxford-IIIT Pet Dataset. <https://www.robots.ox.ac.uk/~vgg/data/pets/>
7. TensorFlow Datasets. <https://www.tensorflow.org/datasets>

8. TensorFlow/Keras Documentation. https://www.tensorflow.org/api_docs